

Embedded C & CAN Interview Preparation

160 Questions & Answers — C Basics, Embedded C, and CAN Protocol

Sriram M

- Section 1: C Basics
- Section 2: Data Types
- Section 3: Variables
- Section 4: Pointers
- Section 5: Arrays
- Section 6: Functions
- Section 7: Memory
- Section 8: Bit Manipulation
- Section 9: Embedded Concepts
- Section 10: Coding
- CAN Interview Questions
 - Basics
 - Advanced
 - Practical
 - Scenario-Based

Section 1: C Basics

Q1. What is Embedded C? Embedded C is a set of language extensions and coding practices built on standard C, used to program microcontrollers and embedded systems. It adds features like direct hardware register access, fixed-width data types, interrupt handling, and low-level I/O, while normally avoiding dynamic memory and OS-dependent libraries.

Q2. Difference between C and Embedded C? Standard C is a general-purpose language that runs on top of an OS with full standard library support (`stdio` , `malloc` , file I/O). Embedded C targets resource-constrained hardware directly (no OS or a small RTOS), emphasizes deterministic execution, avoids heap allocation where possible, and adds hardware-specific keywords/pragmas (e.g., `volatile` for registers, `sfr` , interrupt attributes) provided by the compiler vendor.

Q3. Why is Embedded C used? Because it gives near-assembly-level control over hardware (registers, memory, timing) while retaining C's readability, portability, and structured programming — making it efficient for microcontrollers with limited RAM/ROM and real-time constraints.

Q4. Compilation process? 1. **Preprocessing** — macro expansion, header inclusion, conditional compilation → `.i` file. 2. **Compilation** — translates preprocessed C to assembly → `.s` file. 3. **Assembly** — assembler converts `.s` to machine code (object file) → `.o` . 4. **Linking** — linker combines object files + libraries, resolves symbols, assigns memory addresses → final executable (`.elf` / `.hex`).

Q5. Preprocessor directives? Instructions to the preprocessor, starting with `#` , processed before compilation: `#include` , `#define` , `#undef` , `#ifdef` / `#ifndef` / `#endif` , `#if` / `#elif` / `#else` , `#pragma` , `#error` .

Q6. Difference between `#define` and `const`? `#define` is a textual macro substitution done by the preprocessor — no type checking, no memory allocated, no scope. `const` creates a real, typed variable with scope and storage, is visible to the debugger, and is type-checked by the compiler. `const` is generally preferred in embedded code for safety.

Q7. Storage classes? `auto` (default local variable), `register` (hint to keep in CPU register), `static` (retains value across calls / file scope), `extern` (declares a variable/function defined elsewhere). They

control storage duration, scope, and linkage.

Q8. Scope of variables? The region of the program where a variable name is visible: **block/local scope** (inside `{ }`), **function scope** (labels), **file/global scope** (declared outside all functions), and **function prototype scope** (parameter names in a declaration).

Q9. Lifetime of variables? How long a variable's storage persists: **automatic** (created/destroyed with each function call — stack), **static** (exists for the entire program run), **dynamic** (heap — exists from `malloc` until `free`).

Q10. What is a header file? A `.h` file containing function prototypes, macro definitions, type definitions, and extern declarations, included via `#include` so multiple source files can share common declarations without duplicating code.

Section 2: Data Types

Q11. Size of int? Typically 2 bytes on 8/16-bit MCUs (e.g., 8051) and 4 bytes on 32-bit MCUs (ARM Cortex-M). It's compiler/architecture dependent — never assume; use `sizeof` or fixed-width types (`stdint.h`) in embedded code.

Q12. Size of char? Always 1 byte (guaranteed by the C standard), representing the smallest addressable unit of memory. It can be signed or unsigned depending on the compiler/platform.

Q13. Signed vs unsigned? Signed types can represent negative and positive values (using two's complement), while unsigned types use the full bit range for non-negative values only, doubling the positive range. Mixing signed/unsigned in comparisons/arithmetic causes implicit conversion and is a common source of embedded bugs (e.g., a negative value becoming huge when compared to unsigned).

Q14. Integer promotion? In expressions, operands smaller than `int` (`char`, `short`, bit-fields) are automatically promoted to `int` (or `unsigned int`) before arithmetic/comparison operations. This can cause subtle bugs, e.g., bitwise operations on `char` variables producing unexpected results after promotion and truncation.

Q15. Enum? A user-defined type consisting of named integer constants, improving readability over raw magic numbers (e.g., `enum State {IDLE, RUNNING, ERROR};`). Underlying storage is implementation-defined but usually `int`.

Q16. Typedef? Creates an alias for an existing type, improving readability and portability — e.g., `typedef unsigned char uint8_t;` or `typedef struct {...} Motor_t;`. Common in embedded code for defining register/struct types and hardware-independent type names.

Q17. Boolean implementation? Standard C (pre-C99) has no native `bool`; embedded code historically used `#define TRUE 1 / #define FALSE 0` with `int` or `unsigned char`. C99 onward provides `stdbool.h` with `bool`, `true`, `false` (mapped to `_Bool`).

Q18. Bit fields? Struct members that occupy a specified number of bits rather than full bytes, useful for mapping hardware registers or packing flags compactly, e.g. `struct { unsigned int flag1:1; unsigned int mode:3; };`. Layout/order is compiler-dependent, so bit-fields aren't portable across compilers for register mapping — masks are often preferred for that reason.

Q19. Structure padding? The compiler inserts unused bytes between structure members so each member is aligned to its natural boundary (e.g., a 4-byte `int` starts at a 4-byte-aligned address), which can increase `sizeof(struct)` beyond the sum of member sizes.

Q20. Alignment? The requirement that a data type be stored at a memory address that's a multiple of its size (or a compiler/architecture-defined boundary), so the CPU can access it in a single bus cycle. Misaligned access can cause a hardware fault or performance penalty on some architectures (e.g., ARM Cortex-M0 with unaligned access disabled).

Section 3: Variables

Q21. Static variable? Declared with `static` ; retains its value between function calls (lifetime = whole program) but keeps local scope if declared inside a function. At file scope, `static` limits linkage to that translation unit only.

Q22. Global variable? Declared outside all functions, visible throughout the file (and other files via `extern`), with lifetime spanning the entire program. Overuse increases coupling and risk of unintended modification — used carefully in embedded systems (e.g., ISR-shared flags marked `volatile`).

Q23. Local variable? Declared inside a function/block; storage class `auto` by default, allocated on the stack, created on entry and destroyed on exit from that block.

Q24. Register variable? Declared with `register` , a hint to the compiler to keep the variable in a CPU register for fast access (e.g., loop counters). Modern compilers largely ignore the hint and optimize automatically; you also cannot take the address of a `register` variable.

Q25. Extern variable? Declared with `extern` to tell the compiler a variable/function is defined in another translation unit, allowing sharing of globals across multiple `.c` files without redefining them.

Q26. Const variable? A variable whose value cannot be modified after initialization; the compiler enforces this at compile time. Commonly used for lookup tables placed in flash/ROM to save RAM.

Q27. Volatile variable? Tells the compiler the variable's value can change unexpectedly outside normal program flow (hardware register, ISR-modified flag, another thread), so the compiler must not cache it in a register or optimize away repeated reads — every access goes to actual memory.

Q28. Static function? A function declared `static` has internal linkage — visible only within its own source file, preventing name clashes and hiding implementation details from other modules.

Q29. Inline function? Declared with `inline` , it's a hint asking the compiler to substitute the function body directly at the call site instead of doing a call, reducing call overhead for small, frequently-used functions (e.g., register read/write helpers). Actual inlining is at compiler discretion.

Q30. Restrict keyword? `restrict` (C99) is a pointer qualifier telling the compiler that, for the pointer's lifetime, only that pointer (or values derived from it) will access the pointed-to object — enabling more aggressive optimization since the compiler can assume no aliasing.

Section 4: Pointers

Q31. Pointer? A variable that stores the memory address of another variable, enabling indirect access/modification of data, dynamic memory handling, and efficient array/struct passing.

Q32. Null pointer? A pointer explicitly set to point to nothing, represented as `NULL` (typically address 0). Dereferencing it is undefined behavior; checking for `NULL` before use is standard defensive practice.

Q33. Void pointer? A generic pointer (`void*`) that can hold the address of any data type but cannot be dereferenced directly without casting to a concrete type first. Used for generic functions like `memcpy` , or generic callback/context parameters.

Q34. Wild pointer? A pointer that has been declared but not initialized, so it holds a garbage/random address. Dereferencing it leads to undefined behavior (crash or memory corruption).

Q35. Dangling pointer? A pointer that still holds the address of memory that has already been freed or gone out of scope (e.g., a local variable's address after the function returned). Using it is undefined behavior; set pointers to `NULL` after freeing to guard against this.

Q36. Double pointer? A pointer to a pointer (`int **pp`), used to modify a pointer's value itself through a function call (e.g., allocating memory inside a helper function), or for arrays of pointers/strings.

Q37. Function pointer? A pointer that stores the address of a function, allowing functions to be called indirectly, e.g. `void (*fp)(int);` . Widely used for callback tables, ISR vector tables, and state-machine

dispatch.

Q38. Pointer arithmetic? Adding/subtracting an integer to a pointer moves it by that many elements of the pointed-to type (not raw bytes) — `ptr + 1` on an `int*` advances by `sizeof(int)` bytes. Only valid within (or one-past) the same array; arithmetic between unrelated pointers is undefined behavior.

Q39. Pointer to pointer? Same concept as a double pointer — a variable holding the address of another pointer variable, used for multi-level indirection (e.g., dynamic 2D arrays, modifying caller's pointer).

Q40. Pointer to const? `const int *p` — the pointer can be reassigned to point elsewhere, but the data it points to cannot be modified through `p`. Contrast with `int *const p` (constant pointer — pointer itself can't change) and `const int *const p` (neither can change).

Section 5: Arrays

Q41. Array vs Pointer? An array is a fixed-size, contiguous block of memory allocated at compile time; a pointer is a separate variable that merely stores an address and can be reassigned. Array names decay to a pointer to their first element in most expressions, but `sizeof(array)` gives the total array size while `sizeof(pointer)` gives just the pointer's size.

Q42. Multidimensional array? An array of arrays, e.g., `int matrix[3][4]`, stored in row-major order in one contiguous memory block. Used for lookup tables (e.g., calibration maps) in embedded firmware.

Q43. Passing array to function? Arrays decay to a pointer to their first element when passed, so the function receives a pointer, not a copy — `sizeof` inside the function will give the pointer size, not the array size, so the array length must be passed separately.

Q44. Character array? An array of `char` used to store strings, terminated by a null character `'\0'`. Distinct from a `char*` pointing to a string literal (which is typically read-only).

Q45. String manipulation? Working with `char` arrays using library functions such as `strcpy`, `strcat`, `strlen`, `strcmp`, `strtok`, or hand-written equivalents in embedded projects that avoid the standard library. Buffer-overflow safety (bounds checking, using `strncpy` / `snprintf`) is critical in embedded code.

Q46. Flexible array member? A C99 feature allowing the last member of a struct to be an incomplete array (`int data[];`), letting a single allocation hold a struct plus a variable-length trailing buffer — useful for variable-length packet/message structures.

Q47. Array decay? The automatic conversion of an array name to a pointer to its first element in most expression contexts (function arguments, pointer arithmetic) — the array “loses” its size information at that point.

Q48. Dynamic array? An array whose size is determined and allocated at runtime, typically via `malloc` / `calloc` / `realloc`. Rare in deeply embedded/bare-metal code due to heap fragmentation risk and lack of a heap on some MCUs; static/fixed-size buffers are preferred.

Q49. Const array? An array declared `const`, whose elements cannot be modified after initialization — commonly placed in flash/ROM (e.g., lookup tables, calibration constants) to conserve limited RAM.

Q50. Array of pointers? An array where each element is itself a pointer, e.g., `char *names[5]` for an array of string pointers, or `void (*isr_table[16])(void)` for an interrupt vector table.

Section 6: Functions

Q51. Call by value? The function receives a copy of the argument's value; modifications inside the function do not affect the caller's original variable.

Q52. Call by reference? C doesn't have true call-by-reference, but it's simulated by passing a pointer to the variable — the function dereferences the pointer to read/modify the caller's original data.

Q53. Recursive function? A function that calls itself, directly or indirectly, to solve a problem by breaking it into smaller sub-problems, with a base case to terminate recursion. Used sparingly in embedded systems due to stack depth/overflow concerns on limited-RAM MCUs.

Q54. Static function? (See Q28) — A function with internal linkage, visible only within its own source file, used to encapsulate helper functions not meant to be part of a module's public API.

Q55. Inline function? (See Q29) — A function whose body the compiler may substitute directly at the call site to eliminate call overhead.

Q56. Callback function? A function passed via a function pointer to another function/module, which invokes it later in response to an event (e.g., a UART "byte received" callback, a UDS routine-completion callback). Enables decoupled, event-driven design.

Q57. Function pointer? (See Q37) — A pointer variable that holds a function's address, enabling indirect calls, callback mechanisms, and jump tables.

Q58. Variable arguments? Functions accepting a variable number of arguments using `<stdarg.h>` (`va_list`, `va_start`, `va_arg`, `va_end`), as in `printf`-style functions. Used cautiously in embedded firmware due to code size and type-safety concerns.

Q59. Library function? A pre-written, reusable function provided by the standard library (or vendor SDK/HAL) — e.g., `memcpy`, `strlen`, or MCU HAL functions like `HAL_CAN_Transmit` — that can be called without reimplementing common functionality.

Q60. Reentrant function? A function that can be safely interrupted mid-execution and called again ("re-entered") before the first call completes — by another ISR or thread — and still produce correct results. Requires avoiding static/global shared state (or protecting it), and using only local/stack variables or properly guarded resources. Critical for functions called from both main code and ISRs.

Section 7: Memory

Q61. Stack? A region of RAM used for automatic (local) variables, function call frames, return addresses, and saved registers. Grows/shrinks automatically with function calls/returns (LIFO); overflow (e.g., deep recursion) is a common embedded failure mode.

Q62. Heap? A region of RAM used for dynamic memory allocated at runtime via `malloc` / `calloc` / `realloc` and released via `free`. Many bare-metal embedded systems avoid or minimize heap use because of fragmentation and non-deterministic allocation time.

Q63. ROM? Read-Only Memory (or Flash acting as ROM) — non-volatile storage holding the program code, constants, and lookup tables that persist across power cycles and normally aren't modified at runtime.

Q64. RAM? Random Access Memory — volatile memory used for variables, the stack, and the heap during program execution; contents are lost on power-off.

Q65. Memory leak? Occurs when dynamically allocated memory (`malloc`) is never freed and the pointer to it is lost, so that memory becomes permanently unusable — a serious concern in long-running embedded systems where it eventually exhausts RAM and causes a crash.

Q66. Fragmentation? Occurs when free heap memory becomes split into many small, non-contiguous blocks after repeated allocation/deallocation, so that even though total free memory is sufficient, no single block is large enough for a new request. A key reason embedded systems avoid dynamic allocation post-init.

Q67. malloc? Allocates a block of uninitialized memory of a given size from the heap and returns a `void*` to it (or `NULL` on failure) — `void *p = malloc(10 * sizeof(int));`.

Q68. calloc? Allocates memory for an array of elements and initializes all bytes to zero, taking element count and element size as two arguments — `int *p = calloc(10, sizeof(int));`.

Q69. realloc? Resizes a previously allocated memory block (grow or shrink), preserving existing contents up to the smaller of the old/new size, and may move the block to a new address — the original pointer must be reassigned to the returned pointer.

Q70. free? Releases memory previously allocated by `malloc / calloc / realloc` back to the heap. Using memory after `free` (use-after-free) or freeing it twice (double-free) is undefined behavior.

Section 8: Bit Manipulation

Q71. Set bit? `x |= (1 << n);` — sets bit `n` of `x` to 1 using OR with a shifted mask, leaving other bits unchanged.

Q72. Clear bit? `x &= ~(1 << n);` — clears bit `n` to 0 by ANDing with the inverted mask, leaving other bits unchanged.

Q73. Toggle bit? `x ^= (1 << n);` — flips bit `n` (0 to 1, or 1 to 0) using XOR with the shifted mask.

Q74. Check bit? `if (x & (1 << n))` evaluates non-zero if bit `n` is set; commonly right-shifted and masked with 1 to get a clean 0/1 result: `(x >> n) & 1`.

Q75. Count set bits? Brian Kernighan's algorithm is the classic efficient method:

```
int countSetBits(unsigned int x) {
    int count = 0;
    while (x) {
        x &= (x - 1); // clears the lowest set bit
        count++;
    }
    return count;
}
```

Q76. Reverse bits?

```
unsigned int reverseBits(unsigned int x) {
    unsigned int result = 0;
    for (int i = 0; i < 32; i++) {
        result = (result << 1) | (x & 1);
        x >>= 1;
    }
    return result;
}
```

Q77. Bit masking? Using a bit pattern (mask) combined with AND/OR/XOR to isolate, set, or clear specific bits of a value — e.g., `status & 0x0F` extracts the lower nibble. Fundamental for register manipulation and CAN ID filtering.

Q78. Left shift? `x << n` shifts bits left by `n` positions, filling vacated low bits with 0 — equivalent to multiplying by 2^n (for values that don't overflow).

Q79. Right shift? `x >> n` shifts bits right by `n` positions. For unsigned types, vacated high bits are filled with 0 (logical shift). For signed types, behavior is implementation-defined — most compilers use arithmetic shift (sign-extend), preserving the sign bit.

Q80. Endianness? The byte order used to store multi-byte values in memory. Little-endian stores the least significant byte first (lowest address); big-endian stores the most significant byte first. Matters when interpreting multi-byte CAN signals or communicating between tools/MCUs with different byte order (e.g., DBC signal byte order).

Section 9: Embedded Concepts

Q81. Interrupt? A hardware or software signal that causes the CPU to suspend normal execution and jump to a special handler routine (ISR) to service an urgent event (timer expiry, CAN message received, GPIO

edge), then resume the interrupted code afterward.

Q82. ISR? Interrupt Service Routine — the function executed when an interrupt occurs. Should be kept short and fast, avoid blocking calls, typically only sets flags or copies data, deferring heavy processing to the main loop or a task.

Q83. Polling? Continuously checking a status flag or register in a loop to detect an event, instead of using interrupts. Simpler but wastes CPU cycles and has non-deterministic latency compared to interrupt-driven design; used when interrupts aren't available/needed or for simple, tight timing loops.

Q84. DMA? Direct Memory Access — a hardware controller that transfers data directly between memory and peripherals (or memory-to-memory) without CPU intervention, freeing the CPU for other work and enabling high-throughput transfers (e.g., ADC sampling, UART/CAN buffers).

Q85. Watchdog? A hardware timer that resets the MCU if not periodically “kicked”/refreshed by software within a timeout window, used to recover the system automatically from software hangs, infinite loops, or firmware faults.

Q86. Register access? Directly reading/writing memory-mapped hardware registers to configure and control peripherals, typically via pointers to fixed addresses or vendor-provided struct/macro definitions (e.g., `GPIOA->ODR |= (1 << 5);`), always with `volatile` qualification.

Q87. Memory mapped IO? A scheme where peripheral control/status registers are mapped into the same address space as regular memory, so the CPU accesses hardware using normal load/store instructions instead of separate I/O instructions.

Q88. Volatile in registers? Hardware registers must be declared `volatile` because their value can change independently of program flow (due to hardware events), preventing the compiler from caching a stale value in a register or optimizing away “redundant” reads/writes.

Q89. Peripheral initialization? The startup sequence configuring a peripheral before use: enabling its clock, setting GPIO alternate functions/pins, configuring mode/speed/ baud-rate registers, enabling interrupts if needed, then enabling the peripheral itself — usually done once at boot via HAL/driver init functions.

Q90. Startup code? Low-level assembly/C code (`startup_xxx.s` , `system_init`) that runs before `main()` : sets up the stack pointer, initializes the `.data` section (copies initialized globals from flash to RAM), zeroes the `.bss` section, sets up the vector table, and configures the clock system, before jumping to `main()` .

Section 10: Coding

Q91. Reverse string

```
void reverseString(char *s) {
    int len = strlen(s);
    for (int i = 0; i < len / 2; i++) {
        char tmp = s[i];
        s[i] = s[len - 1 - i];
        s[len - 1 - i] = tmp;
    }
}
```

Q92. Reverse integer

```
int reverseInt(int n) {
    int rev = 0;
    while (n != 0) {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    return rev;
}
```


Q93. Palindrome

```
int isPalindrome(char *s) {
    int i = 0, j = strlen(s) - 1;
    while (i < j) {
        if (s[i++] != s[j--]) return 0;
    }
    return 1;
}
```

Q94. Fibonacci

```
void fibonacci(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        printf("%d ", a);
        int next = a + b;
        a = b;
        b = next;
    }
}
```

Q95. Prime number

```
int isPrime(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}
```

Q96. Linked list

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;

void insertFront(Node **head, int val) {
    Node *n = malloc(sizeof(Node));
    n->data = val;
    n->next = *head;
    *head = n;
}
```

Q97. Circular buffer

```
#define BUF_SIZE 8
typedef struct {
    uint8_t buf[BUF_SIZE];
    uint8_t head, tail;
} CircBuf;

int cb_push(CircBuf *cb, uint8_t val) {
    uint8_t next = (cb->head + 1) % BUF_SIZE;
    if (next == cb->tail) return -1; // full
    cb->buf[cb->head] = val;
    cb->head = next;
    return 0;
}
```

A fixed-size buffer that wraps around using modulo indexing — ideal for ISR-to-main-loop data transfer (e.g., UART/CAN RX buffering) without dynamic memory.

Q98. Queue

```
#define Q_SIZE 10
typedef struct {
    int data[Q_SIZE];
    int front, rear, count;
} Queue;

void enqueue(Queue *q, int val) {
    if (q->count == Q_SIZE) return; // full
    q->data[q->rear] = val;
    q->rear = (q->rear + 1) % Q_SIZE;
    q->count++;
}
```

FIFO (first-in-first-out) data structure — used for message queues and task queues in RTOS-based embedded systems.

Q99. Stack

```
#define STACK_SIZE 10
typedef struct {
    int data[STACK_SIZE];
    int top;
} Stack;

void push(Stack *s, int val) {
    if (s->top >= STACK_SIZE - 1) return; // full
    s->data[++s->top] = val;
}

int pop(Stack *s) {
    return s->data[s->top--];
}
```

LIFO (last-in-first-out) data structure.

Q100. Ring buffer Same underlying concept as Q97 (circular buffer) — a fixed-size, wrap-around buffer commonly implemented with `head` / `tail` indices for producer-consumer scenarios like CAN/UART RX/TX FIFOs, distinguishing “empty” vs “full” either by sacrificing one slot or keeping a separate count.

CAN Interview Questions

Basics

Q1. What is CAN? Controller Area Network — a robust, multi-master serial bus protocol designed for real-time communication between ECUs in vehicles/machinery, using differential signaling (CAN_H/CAN_L) for noise immunity, without needing a central host.

Q2. Why CAN? It offers high noise immunity (differential signaling), built-in error detection/handling, message-based (not address-based) multi-master arbitration without data collision, and reduces wiring compared to point-to-point connections — critical for harsh automotive electrical environments.

Q3. CAN architecture? A multi-master, message-broadcast bus where all nodes are connected in parallel on a twisted pair (CAN_H/CAN_L), terminated at both ends with 120-ohm resistors. Every node “hears” every message; each node’s CAN controller + transceiver handles bit-level transmission, arbitration, and error detection, passing valid frames up to the application via filters.

Q4. Standard frame? CAN 2.0A frame with an 11-bit identifier, allowing up to 2048 unique IDs, used for basic CAN networks — structure: SOF, 11-bit ID, RTR, control field, up to 8 data bytes, CRC, ACK, EOF.

Q5. Extended frame? CAN 2.0B frame with a 29-bit identifier (11-bit base ID + 18-bit extension), allowing over 536 million unique IDs, distinguished from standard frames by the IDE bit being set to 1.

Q6. CAN FD? CAN with Flexible Data-rate — an extension supporting up to 64 data bytes per frame (vs 8 in classic CAN) and a higher bit rate during the data phase, while keeping the arbitration phase at the classic rate for backward compatibility. Improves throughput for modern ECU data needs.

Q7. Arbitration? The non-destructive, bitwise contention-resolution method where multiple nodes transmitting simultaneously compare their identifier bit-by-bit on the bus; a node sending a recessive (1) bit while another sends dominant (0) loses arbitration and backs off, so the frame with the lowest ID (highest priority) wins without any data loss or retransmission delay.

Q8. Bit stuffing? After 5 consecutive identical bits, the transmitter automatically inserts one bit of the opposite polarity to ensure enough transitions for receiver clock synchronization (NRZ encoding needs edges); receivers automatically remove stuffed bits. Violation of this rule is detected as a stuff error.

Q9. CRC? Cyclic Redundancy Check — a 15-bit checksum computed over the frame contents and appended in the CRC field; each receiver recomputes the CRC and compares it, flagging a CRC error if there's a mismatch, ensuring data integrity.

Q10. ACK? Acknowledge bit — the transmitter sends this bit recessive, and any receiver that correctly received the frame (valid CRC) overwrites it with a dominant bit during that slot, confirming at least one node received the message correctly. A missing ACK (still recessive) indicates no receiver acknowledged it.

Advanced

Q11. Error frame? A frame transmitted by a node upon detecting a bus error (bit, stuff, CRC, form, or ACK error), consisting of an error flag (6 dominant bits, violating stuffing rules to alert all nodes) followed by an error delimiter, causing all nodes to discard the current message.

Q12. Overload frame? A frame a node transmits to request extra delay between messages, e.g., when it needs more time to process the previous frame before the next one arrives, or in response to a dominant bit detected during intermission.

Q13. Remote frame? A frame with the RTR (Remote Transmission Request) bit set and no data payload, used by a node to request data from another node that owns a particular identifier — the addressed node responds with the corresponding data frame.

Q14. Bus Off? A fault-confinement state entered when a node's Transmit Error Counter (TEC) exceeds 255, in which the node electrically disconnects itself from the bus and stops all transmission/reception until it's reset (typically via 128 occurrences of 11 consecutive recessive bits, or a controller reset) — protects the healthy bus from a persistently faulty node.

Q15. Error Passive? A fault-confinement state entered when either the Transmit Error Counter (TEC) or Receive Error Counter (REC) exceeds 127; the node can still transmit/receive but must send passive error flags (recessive, less disruptive) instead of active ones, and must wait extra time before transmitting again.

Q16. Error Active? The normal, healthy fault-confinement state (both TEC and REC below 128) where a node can transmit active error flags (dominant bits) that actively disrupt the bus to signal an error to all nodes — the default state at power-up.

Q17. Baud rate? The bit transmission rate of the CAN bus (e.g., 125 kbps, 250 kbps, 500 kbps, 1 Mbps), determined by the bit time configuration (propagation segment, phase segments, sync segment) and must be identical for all nodes on the bus.

Q18. Sample point? The specific point within a bit time (expressed as a percentage) at which each node samples the bus level to determine the bit's value, typically placed around 75-87.5% into the bit time to allow signal propagation and settling time before sampling.

Q19. Termination resistor? A 120-ohm resistor placed at each of the two physical ends of the CAN bus to match the cable's characteristic impedance, preventing signal reflections that would corrupt data — a healthy standard CAN bus measures about 60 ohms (two 120-ohm resistors in parallel) between CAN_H and CAN_L.

Q20. CAN physical layer? Defined by ISO 11898-2 (high-speed CAN) — uses a twisted-pair differential bus (CAN_H, CAN_L); dominant bit (logical 0) drives CAN_H high/CAN_L low (larger voltage difference), recessive bit (logical 1) lets both lines float near a common voltage (smaller/no difference), giving strong noise immunity.

Practical

Q21. Explain one CAN frame. A standard data frame consists of: Start of Frame (1 dominant bit), 11-bit Identifier, RTR bit, Control field (IDE, reserved bit, 4-bit DLC specifying 0-8 data bytes), Data field (0-8 bytes), 15-bit CRC + CRC delimiter, 2-bit ACK slot + delimiter, 7-bit End of Frame, and Intermission (3 bits) before the next frame.

Q22. How did you debug CAN? Typical workflow: connect a CAN analyzer/PCAN-USB to tap the bus, capture live traffic to check for the expected IDs/periodicity/data, verify bus loading and error counters, check termination resistance and bus voltage levels with a multimeter/scope if frames are corrupted, and correlate suspect frames against the DBC file to decode signals and pinpoint the faulty ECU or wiring issue.

Q23. PCAN? PEAK-System's PCAN-USB — a common USB-to-CAN interface adapter used with PC software (PCAN-View, or custom tools) to monitor, send, and log CAN traffic during development and diagnostics; it's a hardware device Sriram has used for flashing/diagnostic tool development.

Q24. BusMaster? A free, open-source CAN bus analysis tool (originally from Robert Bosch) used to monitor, transmit, log, and simulate CAN traffic, supporting DBC-based signal decoding and scripting for test automation.

Q25. CANalyzer? A Vector Informatik tool for analyzing, monitoring, and testing CAN (and other automotive bus) traffic — widely used in the industry for advanced tracing, DBC-based decoding, diagnostics (UDS) tracing, and network simulation.

Q26. CANoe? Vector's more advanced ECU development and test tool, extending CANalyzer with simulation of complete networks/ECUs, automated test execution (CAPL scripting), residual bus simulation, and diagnostic (UDS/OBD) testing — commonly used in Tier-1 validation environments.

Q27. Acceptance filter? Hardware in the CAN controller that compares incoming message IDs against configured filter/mask registers, allowing only messages matching the criteria to be stored in the receive buffer/FIFO and interrupt the CPU, reducing software overhead from irrelevant traffic.

Q28. Mask? Used together with a filter ID — bits set to 1 in the mask mean "this bit must match the filter exactly," while bits set to 0 mean "don't care." This allows filtering a whole range of related IDs with one filter/mask pair instead of listing every ID individually.

Q29. CAN IDs? The identifier field of a CAN frame that both indicates message content/type and determines bus priority during arbitration — lower numeric ID value means higher priority (more dominant bits early in arbitration win the bus).

Q30. Priority? Determined purely by the identifier value during arbitration — the frame with the numerically lowest ID (most dominant leading bits) always wins bus access first, ensuring high-priority messages (e.g., safety-critical ABS/brake signals) are never delayed by lower-priority ones.

Scenario-Based

Q31. CAN Bus Off. Indicates a node's Transmit Error Counter exceeded 255 due to sustained transmission errors — investigate wiring/short circuits, missing termination, a faulty transceiver, ground offset issues, or another node stuck dominant; check error counters and error frame history on the analyzer, then reset the node's CAN controller once the root cause is fixed.

Q32. Missing ACK. Means no other node acknowledged receipt — possible causes: the transmitting node is isolated/disconnected from the rest of the bus, no other node is powered/listening, a receiver's acceptance filter excludes that ID, or a wiring fault between transmitter and receivers. Check bus connectivity and confirm at least one other live node is present and correctly filtered.

Q33. CRC Error. The receiver's computed CRC didn't match the transmitted CRC field, indicating data corruption in transit — often caused by electrical noise, poor termination/reflections, marginal signal levels, or a bit error during transmission. Check bus signal integrity (scope), termination, and shielding/grounding.

Q34. Stuff Error. Detected when 6 consecutive identical bits appear on the bus, violating the bit-stuffing rule — usually indicates a bit-level corruption (noise, glitch) or a node stuck dominant/recessive on the line.

Q35. Form Error. Occurs when a fixed-format bit field (e.g., CRC delimiter, ACK delimiter, EOF) doesn't

have its expected recessive value — often points to timing/synchronization issues or a malfunctioning transceiver/controller.

Q36. Bit Error. Detected when a transmitting node monitors the bus and reads back a bit different from what it sent (outside the arbitration/ACK phases where this is expected) — indicates a bus-level electrical problem: short circuit, wiring fault, or bus contention.

Q37. No response. For a diagnostic/UDS request with no reply — check whether the request was sent on the correct CAN ID/channel, whether the target ECU is powered and its filters are configured for that ID, session/security-access prerequisites, and use the analyzer to confirm the request actually appeared on the bus.

Q38. Wrong ID. A message appears with an unexpected identifier — check the DBC/ID mapping for a possible mismatch between what firmware sends and what's expected, verify no ID collision with another ECU, and confirm filter/mask configuration on the receiving node.

Q39. Lost arbitration. A node loses arbitration when it sends a recessive bit but reads back a dominant bit during the arbitration field — this is normal bus behavior (a higher-priority message is winning), not an error; the losing node automatically stops transmitting and retries after the bus is free.

Q40. Debugging steps. General approach: (1) reproduce the issue while capturing bus traffic on an analyzer, (2) check error counters/frames and bus load, (3) verify physical layer — termination, voltage levels, wiring/connectors, ground — with a multimeter/scope, (4) cross-check captured frames against the DBC for correct IDs/DLC/signal values, (5) isolate nodes one at a time if a faulty ECU is suspected, and (6) correlate timing with software logs/traces on the ECU under test.